

GLP Networking API Specification

August 4, 2026

1 Introduction

Multiagent deterministic GLP (madGLP) [?] is a distributed runtime for Grassroots Logic Programs (GLP) [?] in which autonomous agents communicate via message passing. The current implementation runs agents as Dart isolates on a single workstation, with an `IsolateManager` routing messages between them [?]. This document specifies the networking API required to replace the `IsolateManager`, enabling madGLP agents to run on separate devices communicating peer-to-peer. The networking API is medium-agnostic: the same interface is used regardless of the underlying communication medium. Two media are specified:

- **BLE** (Bluetooth Low Energy): for communication between physically proximate devices, enabling offline, infrastructure-free operation.
- **IP** (Internet Protocol): for communication between devices at arbitrary distance, using UDX streams over UDP with rendezvous agents for signaling and NAT traversal.

BLE serves as the bootstrap medium where agents first meet face-to-face and establish trust. IP extends that connectivity beyond physical proximity. The networking layer selects the available medium transparently; the madGLP runtime is unaware of which medium carries a given message. The application-level protocols—the social graph [?], friendship, introduction—are implemented entirely in GLP agent code and are not part of this networking specification. To the networking layer, all madGLP messages are opaque payloads.

2 Networking API

This section specifies the interface between the madGLP runtime and the networking layer. The interface is independent of the underlying communication medium.

2.1 Identity

Each agent has an Ed25519 key pair. The 32-byte public key serves as the agent’s globally unique identity, used for addressing, authentication, and signing. Identity is persistent across sessions.

Function	Signature	Description
Generate identity	<code>putIdentity()</code>	Generate and persist an Ed25519 key pair.
Retrieve identity	<code>getIdentity() → PubKey</code>	Return the agent's public key.

2.2 Peer Discovery and Connection

The networking layer notifies the runtime when peers become reachable or unreachable, regardless of the underlying medium.

When two agents cannot discover each other via BLE proximity, they may exchange identities out-of-band using `generatePeerLink()` and `consumePeerLink(uri)`. A peer link is a signed, time-limited bearer invitation: possession of a valid unused link authorizes one first-contact attempt by the holder.

Function	Signature	Description
Scan for peers	<code>getPeers() → List<Peer></code>	Discover currently reachable peers.

Function	Signature	Description
Peer discovered	<code>onPeerDiscovered(cb)</code>	Callback when a new peer is detected. The callback receives the peer’s public key and nickname. Discovery precedes connection. A peer’s public key and nickname are learned from the signed ANNOUNCE, which is sent in the clear (authenticated by an Ed25519 signature, not encrypted) and bootstraps the identity that the encrypted session is keyed on — so it necessarily arrives <i>before</i> the Noise handshake completes. Since <code>onPeerConnected</code> is gated on an authenticated Noise session (Section 6.1), there is a real “discovered but not yet connected” window spanning the handshake. The Dart implementation therefore fires <code>onPeerDiscovered</code> on first ANNOUNCE receipt (once per identity, registering the agent↔pubkey mapping) and <code>onPeerConnected</code> only once the session authenticates. A peer that leaves and re-enters range re-fires <code>onPeerConnected</code>, not <code>onPeerDiscovered</code>: its identity is already known. (BLE scanning surfaces a device by its derived service UUID even earlier, but that carries neither public key nor nickname, so it cannot drive this callback.)

Function	Signature	Description
Peer connected	<code>onPeerConnected(cb)</code>	Callback when a peer becomes reachable for communication.
Peer disconnected	<code>onPeerDisconnected(cb)</code>	Callback when a peer is no longer reachable.
Check reachability	<code>isPeerReachable(pk) → bool</code>	Whether the given peer is currently reachable via any medium.
Connectivity status	<code>onConnectivityStatus(cb)</code>	Callback when the networking layer detects a change in the agent's public IP address. Triggers the rendezvous protocol (Section 7). Fired on startup and on address change. The Dart implementation names this <code>onConnectivityStatusChanged</code> — the spec name reads like a status getter, but this is a change-event callback. It receives (<code>oldAddress</code>, <code>newAddress</code>) as <code>ip:port</code> strings, either of which may be <code>null</code> when no public address is available; the kind of change is derived from null comparisons (startup or gain: <code>old=null</code>, <code>new!=null</code>; loss: <code>old!=null</code>, <code>new=null</code>; update: both non-null and different). Consider adopting this name and signature in the spec.
Generate peer link	<code>generatePeerLink() → URI</code>	Generate a signed, one-time, RV-mediated peer-link invitation for out-of-band sharing. Full semantics are specified in Appendix D.

Function	Signature	Description
Consume peer link	<code>consumePeerLink(uri) → PubKey</code>	Verify and redeem an incoming peer-link invitation through a listed rendezvous server, register the inviter’s identity, and attempt first contact. Returns the inviter’s public key; see Appendix D.

2.3 Cold Calls and Trusted Environment Levels

A *cold call* is a first-contact connection attempt between two agents that do not already share an active grassroots networking communication channel. Cold calls may be initiated by a nearby BLE encounter or by an explicit deep-link shared out-of-band, requiring IP connectivity (Section 6.2). In the IP case, the inviting agent need not know the redeemer’s public key before the link is opened. A cold call is implicitly a friendship request: accepting the cold call establishes friendship.

Agents may enforce one of two cold-call trust levels, affecting BLE encounters:

- **Open:** The agent surfaces unsolicited cold calls from nearby BLE peers as friendship requests for the user to accept or decline.
- **Closed:** The agent ignores unsolicited cold calls from nearby BLE encounters without surfacing them.

Explicit cold-calls through deep-links shared out-of-band are not affected by the different trust levels: the link is itself an explicit invitation by its creator, so the resulting friendship request is always surfaced for the user to accept or decline regardless of the configured trust level. The invite link includes authorization: it contains a temporary, one-time 256-bit token generated by its creator that is consumed upon usage; see Appendix D for the whole invite flow.

In the **Open** trust level, an agent responds to any nearby Grassroots BLE encounter by completing the ANNOUNCE exchange, thereby revealing its full public key and allowing first contact. In the **Closed** trust level, an agent does not complete ANNOUNCE with an unknown nearby device: it uses the derived UUID to recognize already-known peers, and ignores unmatched BLE encounters.

2.4 Point-to-Point Communication

The networking layer provides reliable delivery of arbitrary byte payloads between peers.

Function	Signature	Description
Send payload	<code>send(pubKey, payload)</code>	Send a byte payload to the peer identified by <code>pubKey</code> .
Receive payload	<code>onReceive(cb)</code>	Register callback for incoming payloads. The callback receives the sender's public key and the byte payload. The Dart implementation additionally exposes a richer <code>onMessageReceived</code> callback with two extra arguments: <code>messageId</code> (a transport-assigned id, useful for dedup and ACK correlation; also dispatched via the <code>Redux MessageReceivedAction</code>) and <code>transport</code> (the peer's currently preferred transport at delivery time, for logging only — not authoritative for the arrival transport). Both <code>onReceive</code> and <code>onMessageReceived</code> fire for every accepted message.

2.5 Networking Assumptions

The networking layer must satisfy the following assumptions for madGLP correctness: **Fair message delivery:** Every message passed to `send()` is eventually delivered to the recipient, assuming the recipient eventually becomes reachable. Messages to temporarily-unreachable peers are queued and delivered when connectivity is restored. **Atomic message delivery:** Each message is delivered as a complete unit. If a message is fragmented across multiple packets, the networking layer reassembles them before delivery. **Integrity:** Messages are not corrupted in transit. The networking layer provides encryption and integrity checks. **Authentication:** The sender's identity (public key) is authenticated. A received message from public key *pk* was indeed sent by the agent holding the corresponding private key.

3 madGLP Communication Model

This section summarizes the aspects of madGLP that are relevant to the networking layer. The full specification is in [?].

3.1 Global Links and Messages

In madGLP, inter-agent communication is realized through *global links*. A global link connects a local variable pair $(X_p, X_p?)$ at agent p to a local variable pair $(X_q, X_q?)$ at agent q . When the writer at one end is assigned, the value is sent to the other end via a message. All inter-agent messages have the form:

$$G := T^\uparrow$$

where G is a *global variable name* identifying the target endpoint, and $T^\uparrow$ is a *globalized term*—a term in which local variables have been replaced by global names for transmission. A global variable name has the form $_w(p, i)$ or $_r(p, i)$, where p is an agent identifier and $i \in \mathbb{N}$ is an index. The name encodes both the originating agent and the role (writer or reader) of the variable that was globalized.

3.2 Message Types

There are two categories of messages, distinguished by the index in the global name: **Regular link messages** (index $i > 0$): A point-to-point assignment sent from the agent where a `global_send` goal fires to the agent that holds the corresponding entry in its global writers table. The message is $G := T^\uparrow$ where G is either $_w(p, i)$ or $_r(p, i)$. Upon receipt, the target agent looks up the entry, assigns the local writer, and removes the entry. **Cold-call messages** (index $i = 0$): A message to an agent’s well-known index-0 serializer, used to initiate communication between agents that share no prior variables. The message is $_w(q, 0) := [T^\uparrow \mid _w(q, 0)]$, wrapping the term in a list cell. Upon receipt, the target agent appends the term to its network input stream and updates (but does not remove) the serializer entry. Multiple agents may cold-call the same target concurrently.

3.3 Message Flow

The lifecycle of a madGLP message involves three steps, corresponding to three madGLP transactions: **Step 1 — Reduce:** A writer X is assigned at agent p . The paired reader $X?$ becomes known, triggering a `global_send(X?, G, Q)` goal. The goal’s reduction globalizes $X?$ ’s value and adds message $(G := T^\uparrow, Q)$ to p ’s outgoing message set M_p . **Step 2 — Send:** The message $(G := T^\uparrow, Q)$ is removed from M_p and placed in the communication channel to Q . *This is the step that the networking layer implements.* **Step 3 — Receive:** The message arrives at agent Q . The runtime looks up G in Q ’s global writers table, localizes $T^\uparrow$ (creating fresh local variable pairs for any global names it contains), assigns the local writer, and reactivates any suspended goals. The networking layer is responsible only for Step 2: delivering a serialized message from agent p to agent Q . Steps 1 and 3 are handled entirely within the GLP runtime on each device.

4 madGLP-to-Networking Mapping

4.1 Agent Identity Mapping

Each madGLP agent identifier (e.g., `alice`, `bob`) is mapped to an Ed25519 public key. The GLP runtime maintains a bidirectional mapping:

- `resolve(agentId) → PubKey`: given a madGLP agent name, return the public key.
- `resolve(pubKey) → AgentId`: given a public key, return the madGLP agent name.

This mapping is established during peer discovery: when agent p discovers agent q via the networking layer, the discovery handshake provides q 's public key, and the runtime registers the association between q 's agent name and public key. For cold-calls, the sender of the network payload must know the target agent's public key. This is obtained either through peer discovery (e.g., BLE proximity) or through out-of-band peer exchange (Section 2.2). In an IP peer-link cold-call, A creates the signed invite, but the holder B becomes the first network sender: the invite supplies A_{pk} , allowing B to address A 's index-0 serializer.

4.2 Message Serialization

A madGLP message ($G := T^\uparrow, Q$) must be serialized into a byte payload for transmission. The payload encodes two components: the global variable name G and the globalized term $T^\uparrow$.

4.2.1 Global Variable Name Encoding

A global name $_w(p, i)$ or $_r(p, i)$ is encoded as:

Field	Size	Description
tag	1 byte	0x01 for $_w$, 0x02 for $_r$
agent	32 bytes	Ed25519 public key of agent p
index	4 bytes	Index i (big-endian unsigned 32-bit integer)

Total: 37 bytes per global name.

4.2.2 Globalized Term Encoding

A globalized term $T^\uparrow$ is a ground term in which all variables have been replaced by global names. It is a standard term (functors, atoms, integers, lists) that may contain embedded global names. The encoding uses a tagged format:

Tag	Value	Encoding
Atom	0x10	Length-prefixed UTF-8 string
Integer	0x11	8-byte signed big-endian

Tag	Value	Encoding
Global name	0x12	37-byte global name encoding (as above)
Functor	0x20	Tag, length-prefixed name, arity (2 bytes), then each argument recursively
List cell	0x21	Tag, head (recursive), tail (recursive)
Empty list	0x22	Tag only

4.2.3 Message Envelope

The complete serialized message is:

Field	Size	Description
version	1 byte	Protocol version (currently 0x01)
msg_type	1 byte	0x01 for regular link, 0x02 for cold-call (index-0 serializer)
global_name	37 bytes	Target global variable name G
term_length	4 bytes	Length of the serialized term in bytes
term_data	variable	Serialized globalized term $T^\uparrow$

4.3 The Send Operation

When the madGLP runtime executes the Send transaction—moving message $(G := T^\uparrow, Q)$ from agent p 's outgoing set M_p to the communication channel—the networking layer performs:

1. **Resolve destination:** Look up the public key for agent Q via `resolve(Q)`.
2. **Serialize:** Encode the message $(G, T^\uparrow)$ into a byte payload according to the format in Section 4.2.
3. **Transmit:** Call `send(pubKey_of_Q, payload)`.

If the destination agent is not currently reachable, the message is queued locally until the peer becomes reachable. This preserves madGLP's fair message delivery assumption (Section 2.5).

4.4 The Receive Operation

When a payload arrives, the networking layer delivers it to the madGLP runtime:

1. **Identify sender:** The `onReceive` callback provides the sender's public key. Resolve to a madGLP agent identifier via `resolve(pubKey)`.
2. **Deserialize:** Decode the byte payload to recover the global variable name G and the globalized term $T^\uparrow$.
3. **Dispatch:** Pass $(G, T^\uparrow)$ to the madGLP Receive transaction, which performs localization and assignment as specified in the madGLP definition.

5 BLE Medium

This section specifies how the networking API (Section 2) is realized over Bluetooth Low Energy.

5.1 BLE Identity and Advertising

Each agent advertises a BLE GATT service whose 128-bit (16-byte) UUID has two parts: a fixed 8-byte *Grassroots prefix* common to all Grassroots agents, and an 8-byte *agent suffix* derived from the agent’s public key (e.g., the first 8 bytes of its SHA-256 hash). The prefix enables general discovery of any nearby Grassroots agent; the suffix enables recognition of a specific agent before connection, by anyone who knows that agent’s public key.

5.2 BLE Discovery

The BLE layer maintains a continuous scan for devices whose service UUID matches the Grassroots prefix. A scan match is only a discovery hint: the advertisement carries neither the agent’s public key nor its nickname, so it does not by itself constitute peer discovery. Upon successful BLE connection, an ANNOUNCE packet is exchanged containing the agent’s full 32-byte public key and a human-readable nickname. `onPeerDiscovered(cb)` fires upon first receipt of a peer’s signed ANNOUNCE — not on a scan match — once per identity (Section 2.2); `getPeers()` reports the currently reachable peers among those so identified. No Bluetooth bonding or pairing confirmation is required; application-level encryption (using the Ed25519 keys) is used instead.

A BLE encounter with an unknown nearby agent is a local cold-call opportunity. Whether the agent accepts new BLE connections or not is governed by the environmental trust levels in Section 2.3.

5.3 BLE Communication

`send(pubKey, payload)` transmits the byte payload over the established BLE GATT connection. The BLE networking layer handles fragmentation and reassembly for payloads exceeding the BLE MTU. `onReceive(cb)` delivers reassembled payloads to the runtime.

6 IP Medium

This section specifies how the networking API (Section 2) is realized over the Internet. Communication uses UDP with UDX (reliable multiplexed streams over UDP) for data transport. NAT traversal is handled by the rendezvous protocol (Section 7).

6.1 IP Connection

Once a UDP path is established (either directly or via hole-punching), agents open a UDX stream over it. UDX provides reliable, ordered, congestion-controlled delivery over UDP, satisfying the networking assumptions (Section 2.5).

Upon establishing the UDX stream, the peers perform a mutual authentication handshake using their Ed25519 keys via the Noise protocol XX pattern (Curve25519 for key exchange, ChaCha20-Poly1305 for encryption, SHA-256 for hashing). The XX pattern transmits each peer’s static public key during the handshake, bound to a proof of possession, and derives forward-secret session keys. Each peer then verifies the delivered key against an expected key: the friend’s known public key from the social graph for reconnection, or—for an IP cold-call—the inviter’s public key from the invite (for the link holder) and the redeemer’s public key from the RV-forwarded claim (for the inviter). A mismatch aborts the connection. The X25519 static key carried by Noise is the birational image of the agent’s Ed25519 identity key (the standard Ed25519→Curve25519 map), so the expected static is computable from the peer’s known Ed25519 public key alone; no separate key distribution is required. Implemented via libsodium’s `crypto_sign_ed25519_pk_to_curve25519 / sk_to_curve25519`: each side recomputes the expected static from the verified sender Ed25519 key and aborts the handshake on mismatch (`NoiseSessionManager._verifyRemoteStatic`). Defence-in-depth: every ANNOUNCE and handshake packet is also Ed25519-signed and verified against its claimed sender, and the Noise session is keyed by that verified identity.

`onPeerConnected(cb)` fires when the UDX stream is established and authenticated. `onPeerDisconnected(cb)` fires when the stream drops.

6.2 IP Cold-Call

Over IP, a cold call is bootstrapped by a deep-link sent out-of-band from inviting agent *A* to some holder *B*. The link is a signed bearer invitation: at creation time *A* need not know *B*’s public key, and possession of a valid unused link authorizes exactly one first-contact attempt.

`generatePeerLink()` produces the link and registers the invite with a chosen rendezvous (RV) server. `consumePeerLink(uri)` verifies *A*’s signature, registers *A*’s identity, and submits a cold-call claim to the RV server named in the link. The RV server validates the claim and forwards *B*’s identity to *A*; if *A* accepts the friendship request, the standard rendezvous protocol (Section 7.4) ensues with the RV server as the signaling agent. Once the UDP path is open, the peers run UDX and Noise XX (Section 6.1), after which the madGLP cold-call payload is delivered to *A*’s index-0 serializer. Full invite mechanics are specified in Appendix D.

6.3 IP Communication

`send(pubKey, payload)` transmits the byte payload over the authenticated UDX stream. Each message is length-prefixed (4-byte big-endian length followed by payload bytes) to provide message framing. `onReceive(cb)` delivers complete, deframed messages to the runtime.

7 Rendezvous

Mobile agents typically run on smartphones behind NAT and do not have static IP addresses. When an agent’s IP address changes (e.g., due to network roaming), all its UDP connections break. This section describes how agents re-establish connectivity, using two complementary

mechanisms: *rendezvous agents* for bootstrap and *friends-based rendezvous* for subsequent reconnections.

7.1 Rendezvous Agents

A *rendezvous agent* is a lightweight GLP agent that runs at a known, static, globally-routable IP address. Its purpose is to coordinate hole-punching between agents that have lost connectivity, and facilitate IP cold-calls. A rendezvous agent:

- Has no friends list and does not participate in the social graph.
- Accepts cold-call connections from any agent.
- Verifies *friendship proofs* (Section 7.1) to confirm that reconnecting agents are friends, and verifies signed invite capabilities for first-contact IP cold-calls (Section 6.2).
- Observes each connecting agent’s public IP address from the networking layer (via the `peer_address/2` system predicate), serving as an address-discovery mechanism.
- Coordinates UDP hole-punching between the two agents by relaying their addresses.

A machine that hosts a rendezvous agent is called a *GLP rendezvous server*. In the terminology of the ICE framework [?], the GLP rendezvous server combines the roles of STUN server (address discovery via `peer_address/2`) and application-specific signaling server (candidate exchange and hole-punch coordination). It does not serve as a TURN relay; relaying, if needed, is handled at the application level via the social graph (Section 7.7).

Any individual or organization can run a GLP rendezvous server. The architecture is federated: agents may use multiple rendezvous servers for redundancy, and the system operates without any rendezvous server when agents are BLE-adjacent or share a connected friend.

While connected, agents share with their friends which rendezvous servers they use (and any preferences or priorities) via ordinary GLP messages. Each agent thus knows, for each friend, which rendezvous servers that friend uses. The purpose of this exchange is convergence: the rendezvous protocol is inherently asymmetric — when agent *A*’s IP changes, both *A* and the disconnected friend *B* must contact the *same* rendezvous agent for the matching to succeed (Section 7.4.1). Since *B* cannot reach *A* directly, *B* must know which rendezvous agents *A* uses, and the only time to learn this is while they are connected.

Friendship proof. When two agents establish friendship, each signs a *friendship attestation*: a statement binding the two agents’ public keys, signed by the attesting agent’s Ed25519 key. An agent can present this attestation to a rendezvous agent to prove that it is a friend of the other agent. The rendezvous agent verifies the signature without needing to know anything about the social graph.

Invite capability. For a first-contact IP cold-call, no friendship attestation exists yet. Instead, the inviter’s signed peer link acts as a one-time bearer capability. The link contains a 256-bit nonce, but the nonce is not sent to the rendezvous agent during redemption. The inviter registers an invite identifier and proof key derived from the nonce; the redeemer derives the same values from the link and sends a MAC proof of possession. The rendezvous agent accepts a redemption only if the link signature is valid, the expiration time has not passed, the invite identifier was registered by the inviter, the proof verifies, and the invite is still unused. A successful redemption consumes the invite atomically, and the rendezvous agent then forwards the redeemer’s identity to the inviter. Accepting the cold call is accepting the friendship request: the inviter’s acceptance establishes friendship and gates the subsequent rendezvous-protocol signaling and Noise handshake.

7.2 Friends-Based Rendezvous

A *friends-based rendezvous* occurs when a common friend that is currently connected to both sides coordinates the hole-punch via ordinary GLP messages. Unlike a rendezvous agent, a friend acting in this role does not have a static IP address (smartphone agents never do), and is not dedicated infrastructure—it is simply a friend that happens to be connected to both parties.

Through the social graph, each agent knows the friends of each of its friends. When agents *A* and *B* lose connectivity, each can identify common friends that may still be connected to both sides and request their help.

Friends-based rendezvous is used after bootstrap: once an agent has re-established its first connection (via a rendezvous agent), that reconnected friend can mediate reconnection to further friends. Friends-based rendezvous requires at least one active connection: when all of an agent’s connections break (e.g., due to an IP change), it cannot reach any friend to request mediation. A rendezvous agent, with its known static address, provides the initial connection from which friends-based rendezvous can proceed. Alternatively, if two agents are BLE-adjacent, friends-based rendezvous can proceed over BLE without any rendezvous agent — BLE provides the initial connection directly.

7.3 Address Maintenance

Each agent’s friends list is extended to include IP addresses: for each friend, the agent records not only the communication channel but also the friend’s current public address. After reconnection, agents notify their friends of their new address via ordinary GLP messages. This information is maintained as part of the social graph.

An agent behind NAT cannot directly observe its own public address. Instead, it learns its address from the rendezvous agent: when an agent connects to a rendezvous agent, the rendezvous agent observes the agent’s public address (the source IP:port of the incoming packets) via the `peer_address/2` system predicate and reports it back. The agent then shares this address with its friends via GLP messages. No external address-discovery infrastructure (such as STUN servers) is needed; the rendezvous agent serves this role.

7.4 Rendezvous Protocol

When agent A 's IP address changes, all of A 's connections break. Reconnection proceeds in two phases: bootstrap via a rendezvous agent, then friends-based rendezvous for the remaining friends.

7.4.1 Bootstrap

Agent A detects the IP change via `onConnectivityStatus` (Section 2.2). Agent B detects that A went silent via `onPeerDisconnected`. Both A and B know which rendezvous agents the other uses (learned while they were connected).

1. A connects to a rendezvous agent S (outbound to S 's known static address—always succeeds through NAT). A sends a cold-call message: *reconnect*(A_{pk} , B_{pk} , *Proof*, *Reply*), where *Proof* is A 's friendship attestation for B and *Reply* is a fresh reply variable.
2. B detects that A disconnected. B sends a UDP packet to each of A 's known rendezvous agents, opening a NAT hole so that the rendezvous agent can reach B . B sends a cold-call message: *available*(B_{pk} , A_{pk} , *Reply*).
3. S verifies A 's friendship proof, observes A 's public address via `peer_address/2` (Section 7.5), observes B 's public address from B 's availability message, and matches A 's reconnect request with B 's availability message. If the proof is invalid, S binds A 's reply to *rejected* and does not store the request. When the match succeeds, S sends B a *punch*($Addr_A$) message via B 's reply variable, and sends A a *punch*($Addr_B$) message via A 's reply variable.

New: the punch is two-sided. The rendezvous agent tells both endpoints to send UDP punch traffic toward the other endpoint, because an outbound mapping from A to S does not in general allow packets from B to reach A through NAT.

4. A and B each receive their *punch*($\cdot$) message and call `punch_udp/1` toward the other's observed public address. Each endpoint sends a short burst of UDP packets, opening the corresponding NAT mappings on both sides. Once the punch packets overlap, a direct UDP path is established.
5. The networking layer chooses a deterministic UDX initiator: the endpoint with the lexicographically smaller public key. The initiator opens a UDX stream over the new UDP path, and the other endpoint accepts it. The stream authenticates via Noise XX (Section 6.1). `onPeerConnected` fires at both agents. Added clarification about who starts the UDX connection to avoid simultaneous-open ambiguity when both sides start a connection: we need one side to listen, and one side to open

If the first rendezvous agent is unreachable or does not respond within a timeout, A and B proceed to the next rendezvous agent on their shared list, ordered deterministically (e.g., lexicographically by public key). Since both sides compute the same ordered list, they converge on the same S .

7.4.2 Friends-Based Rendezvous

After bootstrap reconnects A to one friend B , friends-based rendezvous reconnects A to the remaining friends. When A 's IP changes, all of A 's connections break, but the connections among A 's friends are unaffected (assuming only A changed address). Each agent maintains a friends-of-friends map via the social graph protocol, so every friend of A knows which other agents are also friends of A .

The protocol proceeds as follows:

1. Bootstrap reconnects A to friend B .
2. B knows (from its friends-of-friends map) which of its own friends are also friends of A . B immediately performs friends-based mediation for every such common friend C : B looks up A 's and C 's addresses in its friends list and sends punch instructions to both sides. No request from A is needed.
3. For friends of A that are not common friends of A and B , A consults its friends-of-friends map. For each such friend E , if A has a reconnected friend D that is also a friend of E , A sends D a mediation request for E . For friends of A for which no reconnected friend can mediate, A contacts the rendezvous server to reconnect to them.

The process terminates when all of A 's friends are reconnected, or when the remaining friends cannot be reached through any available mediator.

A single mediation step works as follows:

1. The mediating friend D looks up A 's and C 's current addresses in its friends list.
2. D sends C : *punch_to*($addr_A$); sends A : *punch_to*($addr_C$). If C is not in D 's friends list, the *punch_to* message carries *not_found* and the recipient takes no networking action.
3. Both sides call *punch_udp*/1; the UDX stream establishes.

7.5 System Predicates

The rendezvous protocol requires the following system predicates, which provide the interface between GLP agent code and the networking layer:

System Predicate	Description
<code>peer_address(PubKey, Addr)</code>	Unifies <code>Addr</code> with the public IP address and UDP port of the connected peer identified by <code>PubKey</code> , as observed by the networking layer from the source address of incoming packets. Available only for peers with an active connection. Used by rendezvous agents to learn the addresses of the agents they are helping. Also serves as the address-discovery mechanism: an agent behind NAT learns its own public address from the rendezvous agent, which observes it via this predicate.
<code>punch_udp(Addr)</code>	Instructs the networking layer to begin sending UDP packets to <code>Addr</code> to open a NAT mapping.
<code>verify_friendship(Proof, A, B, Result)</code>	Verifies that <code>Proof</code> is a valid friendship attestation proving that agents <code>A</code> and <code>B</code> are friends. Binds <code>Result</code> to <code>ok</code> or <code>invalid</code> . Used by rendezvous agents.

All other aspects of the rendezvous protocol—the pending request table on S , the coordination logic, the friends-based traversal strategy, the address maintenance in the friends list—are implemented as ordinary GLP agent code.

7.6 GLP Implementation

The rendezvous protocol has been partially implemented and verified in GLP. We describe what has been implemented, what remains, and what is needed to complete the implementation.

What has been implemented. Three components have been implemented in GLP, pass SRSW checking and type checking, and have been verified with comprehensive test suites (presented in full in the appendices):

- **Rendezvous agent** (Appendix A): the server-side agent that processes cold-call *reconnect* and *available* messages, verifies friendship proofs, observes connecting agents’ addresses via `peer_address/2`, maintains a pending request list, and matches requesting pairs. Invalid proofs are rejected via the reply variable. The key mechanism is the *reply-variable chain*: reply variables embedded in cold-call messages are threaded through writer→reader handoffs and assigned when both sides are matched, propagating values back to the original senders.
- **Rendezvous client** (Appendix B): the client-side building blocks used by agents to construct cold-call messages with embedded reply variables and to dispatch on the replies (*punch* or *rejected*). An end-to-end test wires client and server together on a shared message stream, demonstrating the full bootstrap protocol in both arrival orders.

- **A single friends-based mediation step** (Appendix C): an extension of the social graph agent in which the friends list is extended to include each friend’s current IP address. The mediating friend uses a fused `lookup_and_send` procedure that, in a single traversal, locates a friend’s entry, prepends a *punch_to* message to that friend’s output stream, and returns the friend’s address. The *mediate* clause calls this procedure twice with cross-references, leveraging GLP’s suspension mechanism to resolve the mutual address dependency. Unknown peers are handled gracefully via an *AddrResult* type (*found(Addr)* or *not_found*).

What remains. The friends-based rendezvous protocol (Section 7.4.2) relies on a *friends-of-friends map*: each agent must know, for each friend, who that friend’s friends are, in order to identify common friends for proactive mediation and to select mediators for explicit requests. This map is not yet implemented in GLP. The friends-of-friends protocol—in which each agent broadcasts friendship updates to all its friends via a shared broadcast stream, and each recipient maintains a local map of observed friendships—has been specified and formally verified (with safety and liveness proofs) in a companion work, but in pseudocode rather than GLP. The current GLP social graph agent tracks only direct friends.

Without the friends-of-friends map, the implemented code covers individual components (the rendezvous agent, the client protocol, and a single mediation step) but cannot drive the full reconnection protocol described in Section 7.4.2.

What is needed. To complete the implementation, the following steps are required:

1. **Friends-of-friends map in GLP.** Implement the friends-of-friends protocol as an extension of the GLP social graph agent: each agent broadcasts friendship changes to all friends and maintains a local map of each friend’s friends and their status.
2. **Proactive mediation by the bootstrap friend.** When *B* reconnects to *A* (via the rendezvous agent), *B* uses the friends-of-friends map to identify all common friends of *A* and *B*, and proactively mediates for each of them.
3. **Explicit mediation requests.** For friends of *A* not covered by *B*’s proactive mediation, *A* uses its friends-of-friends map to identify a reconnected friend that can mediate, and sends an explicit request.
4. **Integration.** Combine the rendezvous client, the friends-based mediation step, the friends-of-friends map, and the mediation logic into the social graph agent, so that reconnection is triggered automatically when `onPeerDisconnected` fires.

7.7 Failure and Fallback

UDP hole-punching succeeds for cone NAT (full-cone, restricted-cone, port-restricted cone) but may fail for symmetric NAT, where the mapped port differs per destination. When hole-punching fails, agents may fall back to relaying messages through the social graph: a connected common friend forwards madGLP messages on behalf of the unreachable peer. Relaying is an application-level concern, implemented as GLP agent code; the networking layer does not relay.

8 Public Identity and Access

Some people wish their identity to be publicly known, the way to access them publicly known, or both. To support this, we define two publishable self-certifying records—identity and access—that a person can post or disseminate outside the grassroots platform, e.g. on a personal website, in an email signature, through social media, or via a dissemination protocol such as Nostr or Scuttlebutt. The two records are independent: a person may publish either, both, or neither. Each record can be realized as an executable URL: when opened, the grassroots app adds the person’s identity or access information directly.

8.1 Identity Record

An *identity record* binds a person’s public key to public information about them. It consists of:

- the public key of the person (natural or legal);
- the public keys of the person’s identity custodians and the supermajority threshold required for identity replacement;
- any information the person wishes to make public (e.g., first and last name, organizational affiliation);
- if this record replaces or updates a prior identity record: a hash of the public record of the duly-signed identity replacement protocol by the identity custodians of the previous identity;
- a signature by the private key of the person.

Identity custodians, the supermajority threshold, and the identity replacement protocol are defined in a companion paper.

8.2 Access Record

An *access record* binds a person’s public key to the rendezvous servers they use. It consists of:

- the public key of the person;
- the addresses of the rendezvous servers the person uses;
- an epoch number;
- a signature by the private key of the person.

Access records carry no expiration. A later record supersedes an earlier one by epoch number: a recipient holding multiple records signed by the same public key retains only the record with the highest epoch number. To change the list of rendezvous servers, the person publishes a new access record with a higher epoch number.

8.3 Publication Choices

Publishing only an identity record makes a person publicly identifiable but exposes no information about where they can be reached. Publishing only an access record makes a person reachable via the rendezvous mechanism of Section 7 to anyone who knows their public key, without revealing any additional information about them. Publishing both makes the person both publicly identifiable and reachable. Publishing neither keeps the person publicly unidentifiable and publicly inaccessible; they remain reachable through friend-mediated introduction, BLE adjacency, and private sharing of their public key and rendezvous servers.

9 Complete API Summary

The following table summarizes the complete networking API. This is the interface that replaces the Dart `IsolateManager`. All functions in Section 2 are medium-agnostic. The system predicates in Section 7.5 support the rendezvous protocol (Section 7).

Category	Function	Direction	madGLP Role
Identity			
Setup	<code>putIdentity()</code>	Runtime → Network	Generate agent key pair
Query	<code>getIdentity()</code>	Runtime ← Network	Retrieve agent public key
Discovery			
Scan	<code>getPeers()</code>	Runtime → Network	Find reachable agents
Notify	<code>onPeerDiscovered(cb)</code>	Runtime ← Network	Register agent-to-pubkey mapping
Connect	<code>onPeerConnected(cb)</code>	Runtime ← Network	Agent is now reachable
Disconnect	<code>onPeerDisconnected(cb)</code>	Runtime ← Network	Agent no longer reachable; queue messages
Reachable	<code>isPeerReachable(pk)</code>	Runtime → Network	Check before send
Connectivity	<code>onConnectivityStatus(c)</code>	Runtime ← Network	Report IP address changes
Gen link	<code>generatePeerLink()</code>	Runtime → Network	Create signed RV-mediated invite URI

Category	Function	Direction	madGLP Role
Use link	<code>consumePeerLink(uri)</code>	Runtime → Network	Redeem invite through RV and import inviter
Communication			
Send	<code>send(pubKey, payload)</code>	Runtime → Network	Implements madGLP Send transaction
Receive	<code>onReceive(cb)</code>	Runtime ← Network	Feeds madGLP Receive transaction
System Predicates (Section 7.5)			
Peer address	<code>peer_address(PubKey, Addr)</code>	GLP ← Network	Observe connected peer’s public address
UDP punch	<code>punch_udp(Addr)</code>	GLP → Network	Open NAT mapping for hole-punching
Verify	<code>verify_friendship(Proc A, B, Result)</code>	GLP ← Network	Verify friendship attestation

9.1 Correspondence to IsolateManager

The following table shows how each `IsolateManager` responsibility maps to networking calls:

IsolateManager	Networking	Notes
Route message to target isolate	<code>send(pubKey, payload)</code>	Resolve agent name to pubkey, serialize, transmit
Deliver to isolate’s incoming stream	<code>onReceive(cb)</code>	Deserialize, dispatch to Receive transaction
Register isolate for agent	<code>onPeerConnected(cb)</code>	BLE/UDX connection replaces isolate spawn
Cold-call to index-0 serializer	<code>send(pubKey, payload)</code>	Same primitive; <code>msg_type</code> field distinguishes regular from cold-call
Global link assignment delivery	<code>send(pubKey, payload)</code>	Same primitive; <code>msg_type</code> field distinguishes

A Rendezvous Agent — GLP Implementation

The rendezvous agent processes cold-call messages on its input stream. A *reconnect* message from agent *A* includes *A*’s public key, friend *B*’s public key, a friendship proof, and a reply

variable. An *available* message from *B* includes both public keys and a reply variable.

The agent verifies the friendship proof via `verify_friendship` and observes each connecting agent's public address via `peer_address` (both body system predicates). It searches a pending list for a matching request from the other side. If found, `complete_rendezvous` constructs the reply values in its head: *A* receives *punch*(*Addr_B*) and *B* receives *punch*(*Addr_A*). If the proof is invalid, the reply is bound to *rejected*. If no match is found, the request is stored in the pending list.

The key GLP mechanism is the *reply-variable chain*: each cold-call message contains a reply variable (a writer held by the sender). As the rendezvous agent processes the message, the reply variable is threaded through writer→reader handoffs. When both sides are matched, the assignments propagate back through the chain to the original senders.

```
SignalReply ::= punch(Constant) ; rejected.
```

```
VerifyResult ::= ok ; invalid.
```

```
SignalMsg ::= reconnect(Constant, Constant, Constant, SignalReply?)
            %% reconnect(A_pk, B_pk, Proof, Reply)
            ; available(Constant, Constant, SignalReply?).
            %% available(B_pk, A_pk, Reply)
```

```
PendingEntry ::= needs(Constant, Constant, Constant, SignalReply?)
               %% needs(A_pk, B_pk, AddrA, ReplyA?)
               ; ready(Constant, Constant, Constant, SignalReply?).
               %% ready(B_pk, A_pk, AddrB, ReplyB?)
```

```
PendingList ::= [] ; [PendingEntry | PendingList].
```

```
procedure rv_agent(Stream(SignalMsg)?, PendingList?).
```

```
rv_agent([reconnect(A, B, Proof, ReplyA?)|In], Pending) :-
    ground(A?), ground(B?) |
    verify_friendship(Proof?, A?, B?, Status),
    peer_address(A?, AddrA),
    handle_verify(Status?, A?, B?, AddrA?, ReplyA, Pending?, Pending1),
    rv_agent(In?, Pending1?).
```

```
rv_agent([available(B, A, ReplyB?)|In], Pending) :-
    ground(A?), ground(B?) |
    peer_address(B?, AddrB),
    find_needs(A?, B?, AddrB?, ReplyB, Pending?, Pending1),
    rv_agent(In?, Pending1?).
```

```
rv_agent([], _).
```

```
procedure handle_verify(VerifyResult?, Constant?, Constant?, Constant?,
                        SignalReply, PendingList?, PendingList).
```

```

handle_verify(ok, A, B, AddrA, ReplyA?, Pending, Pending1?) :-
    find_ready(A?, B?, AddrA?, ReplyA, Pending?, Pending1).
handle_verify(invalid, _, _, _, rejected, Pending, Pending?).

procedure find_ready(Constant?, Constant?, Constant?, SignalReply,
    PendingList?, PendingList).
find_ready(A, B, AddrA, ReplyA?, [ready(B2, A2, AddrB, ReplyB?)|Ps], Ps?) :-
    A? == A2?, B? == B2? |
    complete_rendezvous(AddrA?, AddrB?, ReplyA, ReplyB).
find_ready(A, B, AddrA, ReplyA?, [P|Ps], [P?|Ps1?]) :-
    otherwise |
    find_ready(A?, B?, AddrA?, ReplyA, Ps?, Ps1).
find_ready(A, B, AddrA, ReplyA?, [], [needs(A?, B?, AddrA?, ReplyA)]).

procedure find_needs(Constant?, Constant?, Constant?, SignalReply,
    PendingList?, PendingList).
find_needs(A, B, AddrB, ReplyB?, [needs(A2, B2, AddrA, ReplyA?)|Ps], Ps?) :-
    A? == A2?, B? == B2? |
    complete_rendezvous(AddrA?, AddrB?, ReplyA, ReplyB).
find_needs(A, B, AddrB, ReplyB?, [P|Ps], [P?|Ps1?]) :-
    otherwise |
    find_needs(A?, B?, AddrB?, ReplyB, Ps?, Ps1).
find_needs(A, B, AddrB, ReplyB?, [], [ready(B?, A?, AddrB?, ReplyB)]).

procedure complete_rendezvous(Constant?, Constant?, SignalReply, SignalReply).
complete_rendezvous(AddrA, AddrB, punch(AddrB?), punch(AddrA?)).

```

The system predicates `peer_address/2`, `verify_friendship/4`, and `punch_udp/1` are described in Section 7.5.

Open: cold-call branch not yet supported. The code above handles only the friend-reconnect form of `reconnect`: every `reconnect` message is fed unconditionally to `verify_friendship/4`, which rejects anything that is not a valid friendship attestation. The IP cold-call flow (Appendix D, step 5), however, has *A* send a `reconnect` whose `Proof` slot carries an invite-ID reference to the pre-registered invite rather than a friendship attestation; the rendezvous agent is expected to dispatch on the proof variant and run a different verifier. Two candidate resolutions, neither applied here:

- Make the `Proof` type polymorphic (friendship attestation vs. invite-ID reference) and have `rv_agent` dispatch on the variant before calling the appropriate verifier.
- Introduce a dedicated `cold_call_accept` message type with its own handler, leaving `reconnect` untouched.

The code in this appendix is left as-is pending that choice.

B Rendezvous Client — GLP Implementation

The client side provides building blocks for agents that need to use a rendezvous agent. `make_reconnect` constructs a *reconnect* cold-call message with a fresh reply variable embedded, returning the message and the paired reply reader. `make_available` does the same for *available* messages. `handle_reply` dispatches on the reply: *punch(Addr)* triggers `punch_udp`; *rejected* requires no networking action.

```
SignalReply ::= punch(Constant) ; rejected.

SignalMsg ::= reconnect(Constant, Constant, Constant, SignalReply?)
              %% reconnect(A_pk, B_pk, Proof, Reply)
              ; available(Constant, Constant, SignalReply?).
              %% available(B_pk, A_pk, Reply)

procedure make_reconnect(Constant?, Constant?, Constant?,
                          SignalMsg, SignalReply).
make_reconnect(A, B, Proof,
               reconnect(A?, B?, Proof?, Reply),
               Reply?).

procedure make_available(Constant?, Constant?, SignalMsg, SignalReply).
make_available(B, A,
               available(B?, A?, Reply),
               Reply?).

procedure handle_reply(SignalReply?).
handle_reply(punch(Addr)) :- ground(Addr?) | punch_udp(Addr?).
handle_reply(rejected).
```

An end-to-end test wires the client and server together on a shared message stream:

```
procedure test_full_scenario(SignalReply, SignalReply).
test_full_scenario(ReplyA?, ReplyB?) :-
    make_reconnect(alice, bob, proof, MsgA, ReplyA),
    make_available(bob, alice, MsgB, ReplyB),
    rv_agent([MsgA?, MsgB?], []).
```

After execution, *ReplyA* is bound to *punch(stub_addr_bob)* and *ReplyB* to *punch(stub_addr_alice)*.

C Friends-Based Rendezvous — GLP Implementation

Friends-based rendezvous is implemented as an extension of the social graph agent. The friends list is extended to include each friend's current IP address: entries have the form *fe(Name, OutputStream, Addr)*. Agents notify friends of address changes via *addr_update* messages.

When agent *A* asks friend *B* to mediate reconnection to *C*, the procedure `lookup_and_send` performs a fused lookup-and-send: in a single traversal of the friends list, it locates a friend's entry, prepends a *punch_to* message to that friend's output stream, and returns the friend's address. The *punch_to* argument carries an *AddrResult*: *found(Addr)* if the peer was located, or *not_found* if not. The recipient dispatches accordingly — `punch_udp` is never called with an invalid address.

The *mediate* clause calls `lookup_and_send` twice with cross-references: the first call sends *A* a *punch_to(AddrC?)* where *AddrC* is not yet bound; the second call binds *AddrC* when it finds *C*'s entry. GLP's suspension mechanism fills in the unbound reader once the writer is bound — the standard futures/promises mechanism.

```
AddrResult ::= found(Constant) ; not_found.
```

```
MsgOp ::= mediate(Constant)           %% mediate(C_pk)
        ; punch_to(AddrResult)       %% punch_to(found(Addr)) or punch_to(not_found)
        ; addr_update(Constant).     %% addr_update(NewAddr)
```

```
Msg ::= msg(Constant, Constant, MsgOp).
MsgStream ::= [] ; [Msg | MsgStream].
FriendStream ::= [] ; [Msg | FriendStream].
FriendEntry ::= fe(Constant, FriendStream?, Constant).
FriendList ::= [] ; [FriendEntry | FriendList].
```

```
procedure fmrv(Constant?, MsgStream?, FriendList?).
```

```
fmrv(Id, [msg(From, To, addr_update(Addr))|In], Fs) :-
    ground(Id?), Id? == To? |
    update_addr(From?, Addr?, Fs?, Fs1),
    fmrv(Id?, In?, Fs1?).
```

```
fmrv(Id, [msg(From, To, mediate(C))|In], Fs) :-
    ground(Id?), ground(From?), ground(C?), Id? == To? |
    lookup_and_send(From?, Id?, AddrC?, AddrFrom, Fs?, Fs1),
    lookup_and_send(C?, Id?, AddrFrom?, AddrC, Fs1?, Fs2),
    fmrv(Id?, In?, Fs2?).
```

```
fmrv(Id, [msg(_, To, punch_to(found(Addr)))|In], Fs) :-
    ground(Id?), Id? == To? |
    punch_udp(Addr?),
    fmrv(Id?, In?, Fs?).
```

```
fmrv(Id, [msg(_, To, punch_to(not_found))|In], Fs) :-
    ground(Id?), Id? == To? |
    fmrv(Id?, In?, Fs?).
```

```

fmrv(_, [], _).

procedure lookup_and_send(Constant?, Constant?, AddrResult?, AddrResult,
                          FriendList?, FriendList).
lookup_and_send(Key, Id, OtherAddr, found(Addr?),
  [fe(K, [msg(Id?, K?, punch_to(OtherAddr?))|Out1?], Addr)|Rest],
  [fe(K?, Out1, Addr?)|Rest?]) :-
  Key? == K?, ground(Addr?) | true.
lookup_and_send(Key, Id, OtherAddr, KeyAddr?,
  [F|Rest], [F?|Rest1?]) :-
  otherwise |
  lookup_and_send(Key?, Id?, OtherAddr?, KeyAddr, Rest?, Rest1).
lookup_and_send(_, _, _, not_found, [], []).

procedure update_addr(Constant?, Constant?, FriendList?, FriendList).
update_addr(Key, NewAddr, [fe(K, Out?, _)|Fs],
  [fe(K?, Out, NewAddr?)|Fs?]) :-
  Key? == K? | true.
update_addr(Key, NewAddr, [F|Fs], [F?|Fs1?]) :-
  otherwise | update_addr(Key?, NewAddr?, Fs?, Fs1).
update_addr(_, _, [], []).

```

D IP Cold-Call Invite Redemption

An IP cold-call peer link is a signed bearer invitation created by agent *A* for an as-yet-unknown holder *B*. The link is base64url-encoded and is redeemed through a rendezvous server.

```

Invite ::= invite(
  A_pk,
  A_nickname,
  RvServer,      %% RV address and public key
  Nonce256,
  ExpiresAt,
  SignatureA
).

```

Nonce256 is 256 uniformly random bits. ExpiresAt is one hour after creation. RvServer contains a rendezvous server's public address and key. SignatureA is an Ed25519 signature by *A* over all preceding fields. The nonce is usable exactly once: *A* records the invite as unused with the rendezvous agent when the link is generated, and the rendezvous agent marks it consumed atomically when validating the redeemer's claim. Possession of the link authorizes a first-contact attempt; friendship is established only when *A* accepts the resulting cold call from *B*.

The link is base64-encoded for compact URI transport, allowing the recipient to open it as a deep-link. The protocol relies on the out-of-band channel to deliver the link to the intended recipient, and relies on the 256-bit nonce, one-hour expiration, and A 's signature to prevent guessing, replay, and tampering.

Both A and any holder of the link derive an opaque invite identifier and proof key:

$$\begin{aligned} \text{InviteId} &= H(\text{"glp invite id"} \mid \text{Nonce256}), \\ \text{InviteKey} &= \text{HKDF}(\text{Nonce256}, \text{"glp invite proof"} \mid A_{pk} \mid RvServer \mid ExpiresAt). \end{aligned}$$

The raw nonce is not sent over the IP network. Network messages use `InviteId` plus MACs under `InviteKey`, so passive observers and off-path attackers cannot learn or guess the nonce from redemption traffic.

D.1 Rendezvous-Gated Redemption

1. **Generation and registration.** A picks a fresh 256-bit secret, computes the invite ID and proof key from it, and assembles the link (public key, nickname, the chosen rendezvous server S 's address and public key, secret, expiration). A signs the link with its private key and records the invite locally as unused until the expiration. Over A 's existing authenticated connection to S , A registers the invite by sending A 's public key, the invite ID, the proof key, and the expiration. S stores these together with an "unused" flag.
2. **Link delivery.** A sends the base64url-encoded link to B out-of-band.
3. **Cold-call claim.** B opens the link. `consumePeerLink(uri)` decodes it, verifies A 's signature, checks that it is still valid, registers A 's public key and nickname locally, and recomputes the invite ID and proof key from the secret. B opens an encrypted, authenticated connection to S (using S 's public key from the link) and submits a cold-call claim: the invite ID, B 's public key, B 's nickname, and a short proof computed from the proof key covering A 's public key, B 's public key, the invite ID, and a fresh nonce of B 's own. B then sends a standard rendezvous `available` message naming itself and A .
4. **Claim validation and forwarding.** S checks four conditions: the invite ID is registered for A ; the expiration is still in the future; the invite is still marked unused; and the proof verifies against the proof key stored in step 1. On any failure, S rejects the claim. On success, S atomically flips the invite to "used" and forwards B 's public key and nickname to A over A 's existing connection to S .
5. **Acceptance.** A presents B 's public key and nickname to the application layer as a friendship request. If A accepts the request (which is the friendship decision), A sends S a standard rendezvous `reconnect` message naming B . In place of the friendship attestation that the reconnect message normally carries, the proof field references the invite ID that A pre-registered with S in step 1; S matches it against its invite state to authorize the request.

6. **Signaling.** *S* now holds *A*'s `reconnect` and *B*'s `available` for the same pair of public keys, and the standard rendezvous protocol (Section 7.4) takes over with *S* as the signaling agent. *S* observes both public addresses from the connections it holds and sends each side the other's address.
7. **UDP path and Noise handshake.** *A* and *B* each send UDP packets toward the address they received. Once the UDP path is open, the side designated as initiator (by a deterministic rule based on the public keys) opens a UDX stream, and both sides run the Noise XX handshake using their Ed25519 keys (Section 6.1). The rendezvous server sees signaling metadata and observed addresses, never the encrypted payload.
8. **Cold-call delivery.** Only after the Noise-authenticated UDX stream is established does *B* send the madGLP cold-call payload, which the networking layer delivers to *A*'s index-0 serializer. The friendship between *A* and *B* was established by *A*'s acceptance in step 5.